National University of Computer and Emerging Sciences
Lahore Campus

Software Testing (CS 4036)

Date: 24 September, 2025

Course Instructor(s):

Mr. M. Waqas

Answer all questions. Answer questions in the same order as they appear on question paper. Answer all questions on answer sheets.

Roll No _____ Section _____ Student Signature _____

Sessional-I Exam

Total Time (Hrs):	**1**
Total Marks:	**25**
Total Questions:	**2**

Question1: Consider the following program and answer the questions.

```
void calculate_factorial(int n)
{
  int result = 1;


  if (n < 0)
  {
     std::cout << "Error: Factorial is not defined for negative numbers." << std::endl;
     return;
  }

  // C. Loop for calculation
  for (int i = 1; i <= n; i++)
  {
     result = result * i;
  }


  std::cout << "The factorial of " << n << " is: " << result << std::endl;
}
```

Exam questions.

1. Draw dataflow graph (definition and use nodes for variables clearly annotated). [10]
2. Identify paths to satisfy all DU path criteria w.r.t. to all variables. [3]
3. Find out DCF(definition clear path in each DU path.) [2]

Solution:

The DFG is built by numbering the executable statements and identifying the **definition (def)** and **use (use)** of each variable.

Node	Code Statement	def	use
1	`int result = 1;`	`result`	
2	`if (n < 0)`		`n`
3	`std::cout << "Error: ..."`		
4	`return;`		
5	`for (int i = 1; i <= n; i++)`	`i`	`n, i`
6	`result = result * i;`	`result`	`result, i`
7	`std::cout << "The factorial of " << n << " is: " << result << std::endl;`		`n, result`

Export to Sheets

DFG Structure

The DFG can be visualized as two main paths originating from Node **2**:

Start→1→2

Path 1: Error Handling (Negative Input)

2→3→4→End

Path 2: Calculation (Non-Negative Input)

2→5→6→5 (Loop Back)→7→End

---

2. DU Paths to Satisfy All-DU-Path Criteria

A **Definition-Use (DU) Path** is a path (d,u) where d is the node that **defines** (assigns a value to) a variable v, and u is the node that **uses** v, and v remains **undefeated** (no new definition) along that path.

Variable	Definition Node (d)	Use Node (u)	DU Path (d,u)	Path Sequence (Nodes)
n	**(Input)**	**2**	n(Input,2)	1→2
n	**(Input)**	**5**	n(Input,5)	1→2→5
n	**(Input)**	**7**	n(Input,7)	1→2→5→7 (For n=0 or loop completes)
result	**1**	**6**	result(1,6)	1→2→5→6
result	**1**	**7**	result(1,7)	1→2→5→7 (For n=0)
result	**6**	**6**	result(6,6)	1→2→5→6→5→6 (Loop iteration)
result	**6**	**7**	result(6,7)	1→2→5→6→7 (Loop completes)
i	**5**	**5**	i(5,5)	1→2→5 (Initialization/condition check)
i	**5**	**6**	i(5,6)	1→2→5→6
i	**(5)**	**5**	i(5,5)	1→2→5→6→5 (Loop update/check)
i	**(5)**	**6**	i(5,6)	1→2→5→6 (From update)

Export to Sheets

Note on i: The definition of i is split between the initialization (`int i = 1`) and the increment (`i++`) within the structure of Node **5**. For simplicity in the control flow, we treat **5** as the sole definer of the loop variable i.

---

3. Definition Clear Path (DCF)

A path is **Definition Clear (DCF)** with respect to a variable v defined at d and used at u if no other node on the path *between* d and u redefines v. Since the DU paths listed in Section 2 are the *shortest, most direct* paths for the given definitions and uses, **all identified DU paths are also Definition Clear Paths (DCFs).**

DU Path (d,u)	Path Sequence (Nodes)	Variable	DCF Path Status
n(Input,2)	1→2	n	DCF
n(Input,5)	1→2→5	n	DCF
n(Input,7)	1→2→5→7	n	DCF
result(1,6)	1→2→5→6	result	DCF
result(1,7)	1→2→5→7	result	DCF
result(6,6)	5→6→5→6	result	DCF (No other definition between the def at 6 and the use at 6)
result(6,7)	5→6→7	result	DCF
i(5,5)	1→2→5	i	DCF
i(5,6)	1→2→5→6	i	DCF
i(5,5)	6→5 (from i update/check)	i	DCF
i(5,6)	5→6 (from i update/check)	i	DCF

Export to Sheets

Question2: Consider the following function code and control flow graph of check_leap_year. Answer the following questions accordingly.

```
void check_leap_year(int year) {

  if (year % 4 == 0) {

    if (year % 100 == 0) {

      if (year % 400 == 0) {
        std::cout << year << " is a leap year." << std::endl;
      } else {
        std::cout << year << " is not a leap year." << std::endl;
      }
    } else {
      std::cout << year << " is a leap year." << std::endl;
    }
  } else {
    std::cout << year << " is not a leap year." << std::endl;
  }
```

```
}
```

1. What is the minimum number of test cases required to perform basis path testing of the program? Show the complete process of finding the number of required test cases. [3]

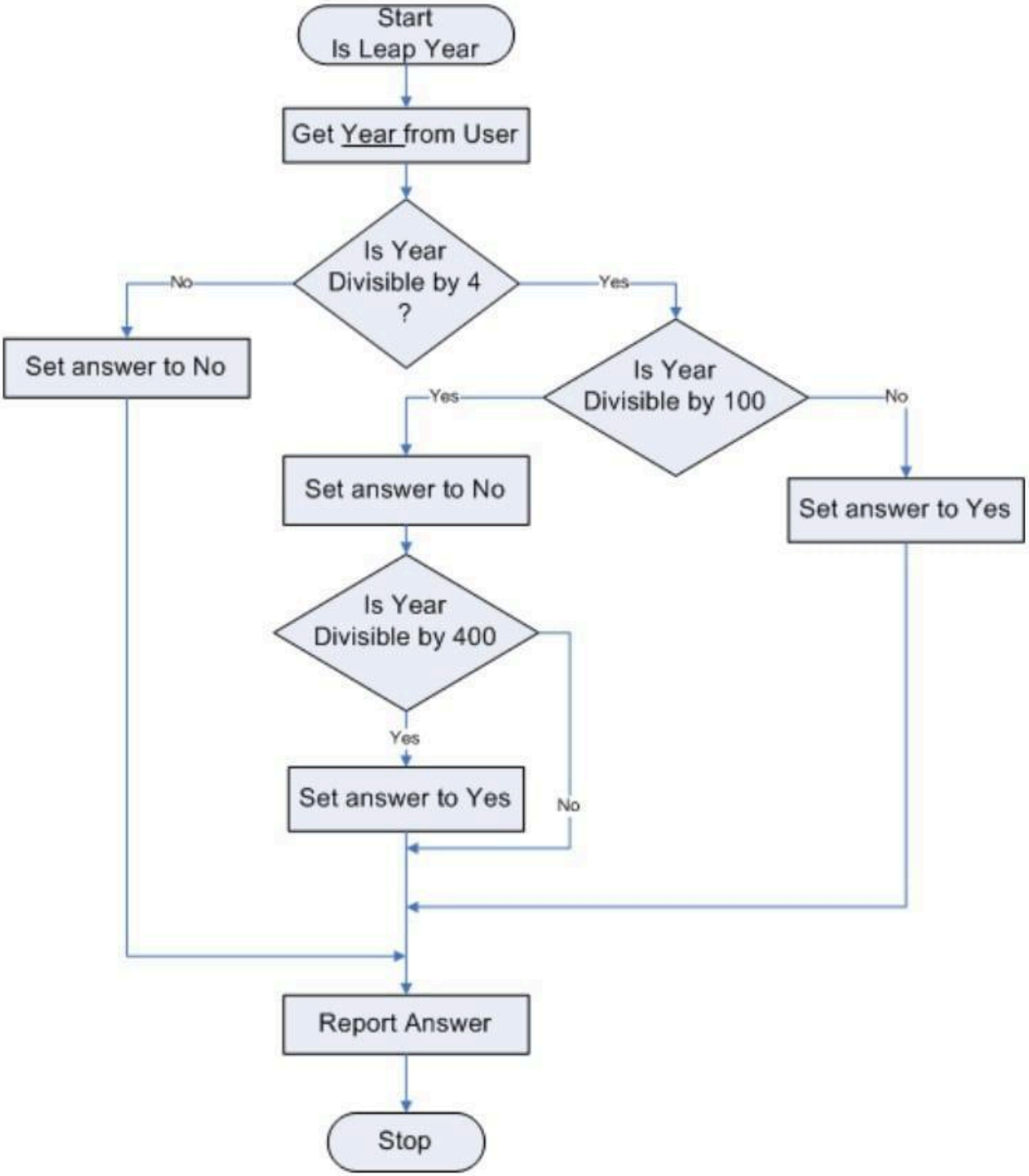
Total Edges E=13

V(G)=13−11+2=4

2. Identify and list the basis paths. [5]
3. Provide at least 2 test cases. [2]

Path Label	Description (Logic)	Path Sequence (Nodes)
Path 1	**Not Divisible by 4** (Year is NOT a Leap Year)	**N1** → **N2** → **N3 (No)** → **N4** → N10 → N11
Path 2	**Divisible by 4, but NOT by 100** (Year IS a Leap Year)	N1 → N2 → N3 (Yes) → **N5 (No)** → **N9** → N10 → N11
Path 3	**Divisible by 4, by 100, and by 400** (Year IS a Leap Year)	N1 → N2 → N3 (Yes) → N5 (Yes) → **N6** → **N7 (Yes)** → **N8** → N10 → N11
Path 4	**Divisible by 4, by 100, but NOT by 400** (Year is NOT a Leap Year)	N1 → N2 → N3 (Yes) → N5 (Yes) → N6 → **N7 (No)** → N10 → N11

Note: CFG is provided on the next page.


Start
Is Leap Year
Get Year from User
Is Year Divisible by 4 ?
No
Yes
Set answer to No
Is Year Divisible by 100
Yes
No
Set answer to No
Set answer to Yes
Is Year Divisible by 400
Yes
Set answer to Yes
No
Report Answer
Stop